

Comparative Evaluation of Large Language Models for Text-to-SQL Semantic Parsing Across Relational Database Engines

ΜΥΛΩΝΑΣ Γεώργιος

Σχολή Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών

Εθνικό Μετσόβιο Πολυτεχνείο,
Αθήνα, Ελλάδα

AM: el21669 – el21669@mail.ntua.gr

ΚΟΤΡΩΝΗΣ Βασίλειος

Σχολή Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών

Εθνικό Μετσόβιο Πολυτεχνείο,
Αθήνα, Ελλάδα

AM: el21432 – el21432@mail.ntua.gr

ΜΑΡΡΕΣ Χρήστος

Σχολή Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών

Εθνικό Μετσόβιο Πολυτεχνείο,
Αθήνα, Ελλάδα

AM: el18912 – el18912@mail.ntua.gr

Ομάδα 34

Abstract— Translating natural language queries into structured query language (Text-to-SQL) is a fundamental challenge in information-based systems, bridging the gap between non-technical domain users and relational databases. In this study, we present *LLMSQL3*, a comprehensive empirical benchmark comparing a commercial closed-weights model (OpenAI GPT-4o-mini) and an open-weights code-specialized model (Qwen2.5-Coder-7B-Instruct, 4-bit quantized) across two widely deployed Relational Database Management Systems (MySQL 8.0 and MariaDB 11.0). Our evaluation corpus comprises 10,575 natural-language/SQL pairs drawn from three established benchmarks (Geography, ATIS, Advising) supplemented by 31 custom-designed and manually verified multi-schema queries designed to stress-test join composition, nested aggregation, and correlated subqueries. From this corpus we draw a sample of 331 questions — a stratified selection of approximately 300 items across the three public domains plus the complete set of 31 custom queries — and measure strict and lenient execution accuracy, execution-error rates, and inference latency for both models on both database engines. Of the 331 sampled items, 244 are scoreable on MySQL and 247 are scoreable on MariaDB (the remainder are excluded on each engine because their reference SQL fails to execute against that engine's schema or returns no rows). GPT-4o-mini achieves 20.5% strict and 25.0% lenient execution accuracy, compared to 14.8% strict and 18.0% lenient accuracy for Qwen2.5-Coder-7B-Instruct. Using an exact two-sided paired McNemar test, GPT-4o-mini's advantage is statistically significant under both the strict metric ($p = 0.0161$) and the lenient metric ($p = 0.0060$). We additionally find that switching the target RDBMS between MySQL and MariaDB leaves accuracy effectively unchanged while introducing a small, dataset-dependent divergence in which reference queries execute successfully, and we document several implementation pitfalls — case-sensitivity of identifiers, RDBMS-specific session variables, and a subtle asymmetry in lenient-metric scoring — that materially affected our own measurements before being diagnosed and corrected.

Keywords— Text-to-SQL, Large Language Models, Natural Language Interfaces to Databases (NLIDB), GPT-4o-mini, Qwen2.5-Coder, MySQL, MariaDB, McNemar Paired Test, Execution Accuracy.

I. INTRODUCTION

Natural Language Interfaces to Databases (NLIDBs) have long represented a core aspiration of information systems and data engineering [1]. The capacity to automatically translate

natural language questions into valid, semantically equivalent Structured Query Language (SQL) statements — a task formalized as Text-to-SQL — democratizes enterprise data access, mitigates the bottleneck of technical query formulation, and enables real-time decision support systems [2].

Historically, Text-to-SQL solutions relied on rule-based parsers, grammar-based intermediate representations, and specialized sequence-to-sequence neural architectures [3]. The emergence of Large Language Models (LLMs) trained on extensive corpora of code and natural text has fundamentally reshaped the field. State-of-the-art closed foundation models (such as OpenAI's GPT series) and open-weights code models (such as Alibaba's Qwen-Coder series) demonstrate remarkable in-context semantic parsing capabilities without requiring full model fine-tuning [4].

Despite these advances, practical deployment of LLM-based Text-to-SQL pipelines within relational enterprise stacks raises three concrete research questions that motivate this work:

- **Model capability and weight paradigm:** how does a lightweight, 4-bit quantized open-weights code model compare against a cloud-hosted proprietary LLM in execution correctness and syntactic robustness, under an identical one-shot prompting protocol?
- **Database engine dialect invariance:** does switching between two closely related but distinct RDBMS engines — MySQL 8.0 and MariaDB 11.0 — introduce measurable divergence in execution accuracy for LLM-generated SQL?
- **Evaluation metric sensitivity:** to what extent do methodological choices in the scoring procedure itself — strict versus lenient result matching, and the handling of vacuously empty results — alter the empirical conclusions drawn about model competence?

Answering these questions required us to build a full experimental harness rather than reuse an existing benchmark script as-is, since no off-the-shelf tool simultaneously supports dual-LLM, dual-RDBMS, paired statistical evaluation. This was built incrementally with several of the

early design decisions turning out wrong and requiring further tuning. In this work, we document this process of trial-and-error, as we treat the diagnosis and correction of these defects as a vital part of this paper.

Section II situates our evaluation methodology relative to prior Text-to-SQL benchmarks and execution-accuracy protocols. Section III describes the pipeline’s five-stage architecture, the composition and sampling of the 10,575-item corpus, the two evaluated models, and the execution strategy. Section IV reports the resulting measurements under strict and lenient execution accuracy, with paired significance testing. Section V discusses what we consider the most instructive findings, including three concrete implementation errors we discovered and corrected. Section VI enumerates the study’s limitations, and Sections VII–VIII disclose our use of AI assistance and the location of all code and data needed to reproduce every reported number.

II. RELATED WORK & EVALUATION METHODOLOGY

A. Semantic Parsing and Text-to-SQL Evolution

Early benchmarks such as GeoQuery (Geography) and ATIS (Air Travel Information System) established the foundations of semantic parsing, emphasizing multi-table joins and compositional logic over small, hand-curated domains [6], [7]. The Advising dataset extends this tradition into an academic-records domain characterized by non-obvious auxiliary tables (e.g., a separate topical-area table that must be joined to answer seemingly simple course-content questions). Modern large-scale benchmarks such as Spider [2] and BIRD [5] further expanded schema complexity and cross-domain generalization requirements. While early evaluations often relied on exact-set string matching of abstract syntax trees, execution-based evaluation has emerged as the more defensible standard, since semantically identical operations can be expressed through syntactically distinct SQL constructs (e.g., an INNER JOIN versus an equivalent correlated subquery) [4].

A separate strand of work asks not how accurate a Text-to-SQL system is but how that accuracy should be measured and reported at all. Benchmarks differ in whether they treat SQL equivalence at the level of the abstract syntax tree, the query plan, or the executed result set, and results computed under one convention are not directly comparable to results computed under another; a system reported as 80% accurate on one benchmark’s metric and 60% on another’s may in fact be identical in capability, with the entire gap attributable to scoring convention. This observation motivates our own emphasis, throughout Sections III–V, on stating exactly what our strict and lenient metrics do and do not credit, rather than reporting a single accuracy number without that context.

B. Execution Accuracy Metrics and the Empty-Result Problem

Let Q denote a natural-language question, S the relational schema, and D the underlying database instance. Let Y_{gold} be the reference SQL query and Y_{gen} the LLM-generated query, with result sets $R_{\text{gold}} = E(Y_{\text{gold}}, D)$ and $R_{\text{gen}} = E(Y_{\text{gen}}, D)$. We define two execution accuracy metrics used throughout this paper:

- **Strict execution accuracy:** scores 1 if and only if R_{gen} and R_{gold} contain identical tuples (as sets, ignoring row

order), including agreement on the number and identity of projected columns.

- **Lenient execution accuracy:** scores 1 if there exists an injective (one-to-one) matching from the tuples of R_{gold} into the tuples of R_{gen} such that every matched R_{gold} tuple is a subset of its matched R_{gen} tuple — i.e., the generated query may project additional columns beyond those strictly required. This matching is one-directional: it verifies that every reference row is accounted for, but does not require every generated row to correspond to a reference row. Consequently, additional generated rows that do not match any reference row are not penalized; we return to this precise scope, and its consequences, in Section VI.

A related methodological question concerns how much in-context guidance a Text-to-SQL model should receive. Retrieval-augmented and multi-shot prompting strategies have been shown to substantially improve execution accuracy on benchmarks such as Spider [4], sometimes by selecting worked examples similar to the target question at inference time. We deliberately adopt a minimal, single fixed example per schema (Section III-B) rather than a retrieval-augmented scheme, since our goal is to compare two models under an identical, easily reproducible prompting protocol rather than to maximize the absolute accuracy of either; we discuss the consequence of this choice for our reported accuracy figures in Section VI-A.

III. SYSTEM ARCHITECTURE & EXPERIMENTAL PIPELINE

A. Pipeline Architecture

The LLMSQL3 experimental framework is organized into five stages: (i) schema extraction, (ii) prompt assembly, (iii) dual-model inference, (iv) dual-database execution, and (v) dual-metric scoring with paired statistical testing. Fig. 1 summarizes the data flow.

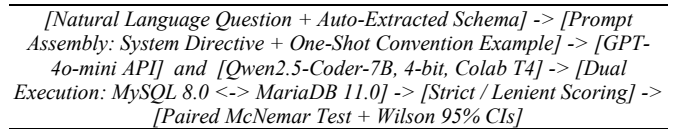


Fig. 1. End-to-end LLMSQL3 evaluation pipeline.

B. Schema Inspection and Prompt Engineering

To avoid biasing the prompt with hand-authored schema descriptions, table and column definitions are extracted at runtime directly from each RDBMS’s information schema (table name followed by its column list). Each prompt further includes a single targeted worked example per source schema (geography, ATIS, advising), establishing the identifier-naming conventions of that specific database instance without providing more examples of the actual evaluation questions. Both models receive an identical system prompt instructing them to output a single valid SQL statement with no surrounding prose or markdown, and both are queried with decoding temperature of zero to maximize determinism and reproducibility.

C. Dataset Composition and Sampling

The evaluation corpus comprises 10,575 question-SQL pairs:

- **Geography** (877 pairs, 7 tables): U.S. geographic entities, borders, rivers, and population statistics.

- **ATIS** (5,280 pairs, ~25 tables): Air Travel Information System, featuring multi-table joins across flights, airports, fares, and restrictions.
- **Advising** (4,387 pairs, 15 tables): academic-records domain with student records, course prerequisites, and instructor loads.
- **Custom queries** (31 pairs, 3 schemas): custom-designed and manually verified queries exercising multi-way joins, nested subqueries, and GROUP BY/HAVING aggregation, distributed 15/8/8 across the Geography, ATIS, and Advising schemas respectively.

A heuristic difficulty label (easy / medium / hard) is assigned automatically to every pair based on structural SQL complexity — table count, subquery nesting depth, presence of aggregate functions, and GROUP BY/HAVING clauses.

For the evaluation reported here we draw a sample of $N = 331$ questions: approximately 300 items stratified by (dataset $\times$ difficulty) from the three public benchmarks, combined with the complete set of all 31 custom queries. The custom queries are included in full, because at their native size they would otherwise contribute only 6-7 items to a 300-question sample, too few to support any dataset-level statistical statement about exactly the queries we ourselves designed to probe compositional SQL generation. Sampling uses a fixed random seed, so the same 331 items are used for every model-database combination reported in Section IV, which is a precondition for the paired significance testing described in Section III-E.

D. Evaluated LLMs and Deployment

- **OpenAI GPT-4o-mini**: accessed via the OpenAI REST API at temperature 0.0, with all other decoding parameters left at their API defaults. Requests are retried with exponential backoff on transient failures to distinguish genuine model errors from network-level failures.
- **Qwen2.5-Coder-7B-Instruct**: an open-weights 7B-parameter code model, quantized to 4-bit precision (NF4 with double quantization, via bitsandbytes) to fit within the 16GB VRAM of a single NVIDIA T4 GPU on Google

Colab, decoded greedily with sampling disabled to match the deterministic behavior requested from GPT-4o-mini. Running the two models on fundamentally different infrastructure, a commercial API versus a self-hosted quantized checkpoint on a free-tier GPU, is part of the comparison we intend to make since it reflects the realistic choice between a proprietary model and a locally deployable open-weights alternative, rather than an attempt to equalize compute budgets between the two.

E. Database Infrastructure and Statistical Testing

Both RDBMS engines are containerized via Docker with identical schemas and row populations: MySQL 8.0 on port 3306 and MariaDB 11.0 on port 3307. Because the generated SQL text does not depend on which engine ultimately executes it, we generate each model's output only once and re-execute the identical SQL against both engines, yielding all four experimental combinations (GPT $\times$ MySQL, GPT $\times$ MariaDB, Qwen $\times$ MySQL, Qwen $\times$ MariaDB) from only two live LLM invocation passes. For hypothesis testing, an item is excluded as unscorable if its reference (gold) query fails to execute against the target schema, or if it executes but returns zero rows; in the latter case the only possible match is a trivial empty match, which is not credited as correct, so the item is a guaranteed failure for every model irrespective of the SQL it generates (Section V-C). Because scoreability is a property of the item and the database, not of the model, the same set of scoreable items is used for both models on a given engine, which we verify programmatically before running the paired test. Wilson score 95% confidence intervals are reported for all accuracy figures, and GPT-4o-mini is compared against Qwen2.5-Coder-7B-Instruct using an exact two-sided paired McNemar test [11], [12].

F. Software and Infrastructure Stack

For reproducibility, Table I lists the exact software components used to build and run the pipeline. All software components are publicly available, open-source, or commercially accessible, and the two RDBMS engines are provisioned as separate Docker containers on non-conflicting host ports so that both can run simultaneously on a single development machine.

TABLE I — SOFTWARE AND INFRASTRUCTURE COMPONENTS

Component	Version / Detail	Role
MySQL	8.0 (Docker), port 3306	Primary RDBMS target
MariaDB	11.0 (Docker), port 3307	Secondary RDBMS target
Python	3.12, pandas, PyMySQL	Pipeline implementation language
OpenAI API	gpt-4o-mini, temperature 0	GPT-side LLM inference
Transformers / bitsandbytes	Qwen2.5-Coder-7B-Instruct, NF4 4-bit	Qwen-side LLM inference
Google Colab	NVIDIA T4 GPU, 16GB VRAM	Qwen inference environment
DBeaver	Community Edition	Manual schema/query verification
Docker	Desktop, containerized isolation	RDBMS provisioning

IV. EXPERIMENTAL RESULTS & STATISTICAL ANALYSIS

A. Overall Performance

Table II summarizes execution accuracy, mean generation latency, and execution-error rate for all four model-database combinations, computed over each engine's own scoreable subset of the 331-item sample (244 items on MySQL, 247 on MariaDB; see Section III-E for the scoreability criterion).

TABLE II — BENCHMARK RESULTS ACROSS LLM ENGINES AND RDBMS TARGETS

Metric	GPT x MySQL	GPT x MariaDB	Qwen x MySQL	Qwen x MariaDB
Strict accuracy	20.5% (50/244)	20.2% (50/247)	14.8% (36/244)	14.6% (36/247)
Lenient accuracy	25.0% (61/244)	24.7% (61/247)	18.0% (44/244)	17.8% (44/247)
Mean gen. latency	1.08 s	1.08 s	4.87 s	4.87 s
Execution errors	36/244 (14.8%)	36/247 (14.6%)	78/244 (32.0%)	80/247 (32.4%)

GPT-4o-mini outperforms Qwen2.5-Coder-7B-Instruct on every metric and is approximately 4.5x faster per query. The item counts scoreable against MySQL and MariaDB differ slightly (244 versus 247) despite the two engines sharing an identical schema; we return to this observation in Section V-D.

Fig. 2. Overall execution accuracy per model, strict vs. lenient; whiskers show 95% Wilson confidence intervals.

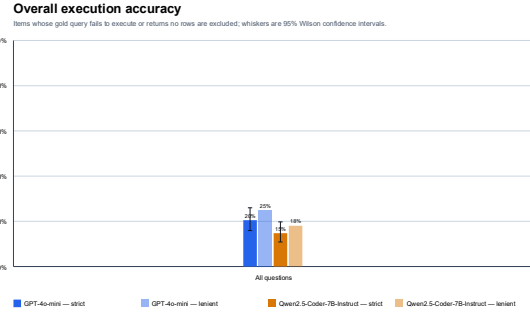
B. Confidence Intervals and Hypothesis Testing

Table III reports the exact paired McNemar comparison between GPT-4o-mini and Qwen2.5-Coder-7B-Instruct over the 244 items scoreable on MySQL.

TABLE III — STATISTICAL COMPARISON (N = 244 SCOREABLE ITEMS)

Metric	GPT-4o-mini (95% CI)	Qwen2.5-Coder-7B (95% CI)	GPT only	Qwen only	Exact p-value
Strict	20.5% [15.9%, 26.0%]	14.8% [10.9%, 19.7%]	22	8	0.0161 (*)
Lenient	25.0% [20.0%, 30.8%]	18.0% [13.7%, 23.3%]	26	9	0.0060 (**)

Under the strict metric, GPT-4o-mini uniquely solves 22 items that Qwen fails, against 8 items solved uniquely by Qwen; the exact McNemar p-value of 0.0161 rejects the null hypothesis of equal error rates at $\alpha = 0.05$. Under the lenient metric the disagreement is larger (26 versus 9) and the effect is even more significant ($p = 0.0060$). Both metrics therefore agree that GPT-4o-mini's advantage is unlikely to be an artifact of sampling variance in this 244-item sample, which



strengthens our confidence in the qualitative conclusion despite the modest accuracy of both approaches.

C. Breakdown Across Domains and Difficulty

Table IV reports the strict/lenient accuracy split across all six dataset categories (three public benchmarks and three custom-query subsets, one per schema).

TABLE IV — ACCURACY BY DATASET (STRICT / LENIENT, MySQL)

Dataset	n	GPT-4o-mini (%/%)	Qwen2.5-Coder-7B (%/%)
Geography	25	64.0 / 68.0	64.0 / 64.0
Advising	57	24.6 / 26.3	12.3 / 15.8
ATIS	135	8.9 / 14.1	3.0 / 5.9
Custom — Geography	15	46.7 / 53.3	40.0 / 53.3
Custom — ATIS	7	14.3 / 14.3	14.3 / 14.3
Custom — Advising	5	0.0 / 20.0	40.0 / 40.0

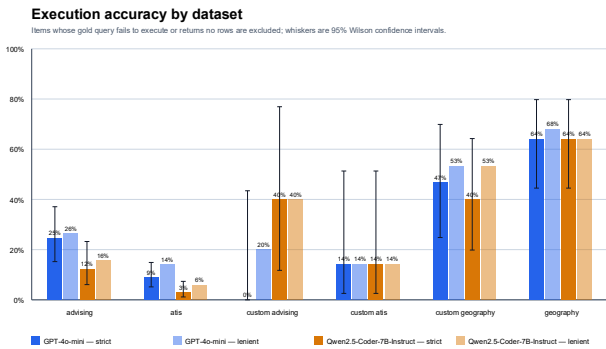


Fig. 3. Execution accuracy by dataset domain, all six categories.

Both models perform strongly and almost identically on Geography (~64%), the structurally simplest of the three public schemas, while both are substantially weaker on ATIS, the most heavily joined schema. The Custom-Advising subset is the one category in which Qwen numerically outperforms GPT-4o-mini (40.0% versus 0.0% strict); given $n = 5$, this reversal falls well within sampling noise, as reflected in the wide confidence intervals in Fig. 3, and should not be read as evidence that Qwen generalizes better to custom-designed compositional queries.

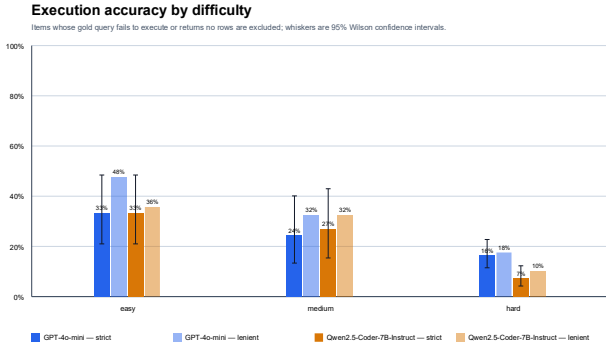


Fig. 4. Execution accuracy by heuristic difficulty tier.

Fig. 4 shows a monotonic pattern for both models: accuracy falls from "easy"-labeled items (33.3% GPT strict, 33.3% Qwen strict) through "medium" (24.3% GPT, 27.0% Qwen) to "hard" (16.4% GPT, 7.3% Qwen). The structural-

V. DISCUSSION & DEEP ERROR ANALYSIS

A. The Strict-versus-Lenient Disparity

On the Advising schema, GPT-4o-mini's accuracy rises only modestly from strict (24.6%) to lenient (26.3%) in the current corrected sample, a much smaller gap than we observed during an earlier stage of this project that we corrected (Section V-C). Manual inspection of the remaining strict/lenient gap across datasets shows it is driven by cases in which the LLM correctly identifies the filtering predicate and join path but projects a different, often larger, set of columns than the reference query. For example, returning `student_id`, `student_name`, `gpa` when the reference strictly projects only `student_id`. This confirms that a purely strict metric can meaningfully understate a model's semantic competence, and that reporting both metrics side by side, as we do throughout this paper, is preferable to reporting either alone.

B. Dataset-Specific Idiosyncrasies

Manual review of ATIS failure cases surfaced two recurring, dataset-specific idiosyncrasies inherited from the original 1990s ATIS corpus [7], both of which are essentially unrecoverable from the natural-language question alone:

- **Hardcoded historical dates:** several reference queries constrain flight dates to specific 1991-1993 calendar dates that are never mentioned in the corresponding question text, reflecting the fixed collection period of the original corpus rather than any information available to a model reasoning from the question alone.
- **Non-literal ground truths:** some reference queries project internal identifiers or airline/airport codes rather than the more natural, human-readable labels a model is likely to produce (for example, favoring an aircraft code over its plain-language description), which strict and even lenient value-matching penalize.

We also identified an intrinsic LLM error pattern where both models occasionally confuse airport codes with city names when a query requires joining through an intermediate airport_service table, producing predicates such as `from_airport = 'ATLANTA'` where the schema instead expects a three-letter airport code such as 'ATL'. A related pattern recurs on the Advising schema: several questions

complexity heuristic we use to assign difficulty labels therefore orders the tiers consistently with the difficulty both LLMs actually experience. We discuss this further in Section VI-D.

D. Generation Latency and Computational Efficiency

Beyond execution accuracy, we separately assess computational efficiency. GPT-4o-mini, accessed via a commercial API backed by data-center-scale infrastructure, averages 1.08 seconds per query. Qwen2.5-Coder-7B-Instruct, run locally under 4-bit quantization on a single shared Colab T4 GPU, averages 4.87 seconds per query — roughly 4.5 times slower. This gap reflects a architectural trade-off between these options. For a production system prioritizing raw response latency, the commercial API is the clear choice under the conditions tested here, whereas for a setting where data residency or per-query cost dominates the decision, the open-weights model remains viable provided the higher execution-error rate documented in Table II is mitigated with a downstream SQL-validation step.

about course content (for example, questions mentioning a topic area such as "artificial intelligence") are only answerable by joining through a separate AREA table that associates courses with topical labels, a join path that is not obvious from the question text or from the course table's own columns. Sometimes, both models instead search the course table's name or description fields directly, which frequently fails to match the relevant rows. We consider this a fair test of a model's ability to reason about implicit schema structure, and it is reflected in the relatively low Advising accuracy in Table IV.

C. A Vacuous-Match Defect in the Lenient Metric

During development we discovered and corrected a subtle logical defect in our own lenient-matching routine: when the reference query returned zero rows, the row-matching loop iterated zero times and the routine returned True unconditionally, regardless of what the generated query returned. Because the strict metric already handled this case correctly (two empty result sets compare as equal only to each other), the defect caused the lenient metric but not the strict metric to systematically over-count correct answers whenever the reference happened to be empty, most visibly on the Advising schema. The fix requires the lenient routine to agree with an empty reference only when the generated result is also empty, mirroring the strict routine's behavior in that special case.

D. Database Dialect Effects

Although MySQL 8.0 and MariaDB 11.0 were provisioned from an identical schema dump, the number of reference (gold) queries that fail to execute differs slightly between the two engines (6 on MySQL versus 3 on MariaDB, out of 331), and this difference is identical for both LLMs, confirming it is a property of the database engine and the reference SQL rather than of either model. We attribute this to minor SQL-dialect divergence between the two engines (for example, differing strictness around reserved keywords or default SQL modes) rather than to any data inconsistency, since the two engines otherwise produce accuracy figures that agree to within one percentage point (Table II). A related, engine-specific issue we encountered is that a MySQL-only session variable used to bound query execution time (MAX_EXECUTION_TIME) is not recognized by MariaDB

so issuing it unconditionally caused every subsequent statement on that connection to fail.

E. A Case Study in Metric-Definition Sensitivity

The final scoreability criterion described in Section III-E, which is excluding an item only when its reference query fails to execute was not our first design choice, and we record the intermediate steps here because we believe the progression is itself instructive. An initial implementation excluded any item whose reference query returned zero rows, reasoning that a model's answer cannot be judged against an empty reference. Applied naively, this criterion removed roughly one quarter of an earlier, smaller 306-item sample, including many items where the generated query returned a clearly incorrect non-empty result against an empty reference, since a non-empty answer to a question whose true answer is "no matching rows" is simply wrong. Removing such items from the denominator inflated strict accuracy by discarding genuine failures rather than genuinely unscorable items, and the effect was large enough to change whether the strict-metric GPT-versus-Qwen difference reached statistical significance. Re-deriving the criterion to exclude only items with a broken reference query restored these genuine failures to the scored sample without reintroducing the vacuous-match problem described in Section V-C, and is the criterion used for every result reported in this paper.

F. Open-Weights Quantized Models versus Closed Cloud Models

While GPT-4o-mini is the stronger model overall, Qwen2.5-Coder-7B-Instruct is numerically competitive with GPT-4o-mini on several small custom-query subsets. For deployment settings in which schema or data cannot leave on-premises infrastructure, a quantized 7B open-weights code model therefore represents a plausible baseline, provided downstream schema-aware SQL validation is layered on top to catch the substantially higher raw execution-error rate we observe for Qwen (32.0% versus 14.8% for GPT-4o-mini).

G. Representative Failure Case Studies

To make the failure patterns summarized above concrete, we walk through four representative items in detail, each illustrating a distinct cause of execution-accuracy failure rather than a repetition of the same mistake.

Case 1: Advising, GPT-4o-mini, strict and lenient both fail. For the question "which courses are related to machine learning," the reference query joins COURSE to the auxiliary AREA table and filters on AREA.AREA LIKE '%machine learning%', reflecting the schema's convention of storing topical labels separately from course records. GPT-4o-mini instead generated `SELECT NAME FROM COURSE WHERE DESCRIPTION LIKE '%machine learning%' OR NAME LIKE '%machine learning%'` — a fluent, syntactically valid query that never joins the AREA table at all. Because most courses whose description mentions the topic are not the same rows tagged in AREA, the two queries return substantially different result sets, and the answer fails under both metrics despite being a reasonable reading of the question in isolation. This is the clearest instance in our corpus of a failure driven by non-obvious schema structure rather than by any weakness in SQL syntax generation.

Case 2: ATIS, both models, strict fails / lenient succeeds. For the question "how many stops does flight AA-201 have," the reference query is `SELECT STOPS FROM FLIGHT`

`WHERE FLIGHT_ID = 'AA-201'`, projecting the stored STOPS column directly. Both models instead generated a query that joins FLIGHT to FLIGHT_STOP and returns `COUNT(*)`, computing the stop count from the leg table rather than reading the pre-aggregated column. On this particular row the two approaches agree numerically, so the lenient metric scores the answer correct once the generated projection is widened to include the raw count, while the strict metric fails it outright because the column identity and projection shape differ from the reference. We include this case because it demonstrates concretely why we report both metrics rather than either alone.

Case 3: Execution error from a non-existent column (ATIS, Qwen2.5-Coder-7B-Instruct, execution error). For the question "what is the cheapest fare from Denver to Boston," Qwen generated `SELECT MIN(FARE_AMOUNT) FROM FARE WHERE FROM_AIRPORT = 'DEN' AND TO_AIRPORT = 'BOS'` — a well-formed query with one defect: the FARE table in this schema stores the cost under `ONE_DIRECTION_COST`, not `FARE_AMOUNT`, and does not itself carry `FROM_AIRPORT/TO_AIRPORT` columns (those live on the FARE's parent FLIGHT row, reached only through a join). The query fails at execution time with an unknown-column error rather than returning a wrong answer, which is characteristic of the execution-error class reported in Table II: unlike Cases 1 and 2, no comparison against the reference is even possible here, and the item is counted as an execution failure rather than a semantic disagreement. Errors of this shape account for a majority of Qwen's execution errors on ATIS, consistent with the higher raw execution-error rate for the smaller, locally hosted model reported in Table II.

The first three cases illustrate three major failure modes discussed above: These are: non-obvious join paths, projection-shape disagreement that a lenient metric can recover, and execution failure from schema misremembering. These mistakes reinforce the finding that aggregate accuracy percentages alone, without this kind of qualitative reading, understate how much of the gap between the two metrics (and between the two models) is attributable to superficial answer-shape mismatch rather than to a failure to understand the question.

Case 4: Custom-Geography, GPT-4o-mini, lenient succeeds. One of our custom-designed queries asks "which states have a coastline longer than 500 miles," with a reference query that joins state to a coastline-length view specific to our custom schema extension and returns only the state name. GPT-4o-mini's generated query correctly identifies the same filtering condition but returns `state_name` together with the coastline length itself. Because the value used for filtering is not the same as the value ultimately displayed, this case is a useful counter-example to the intuition that lenient scoring is simply "generous": the metric still requires the reference's actual projected value (in this case the state name) to appear in the generated row, which it does, but it does not credit the model for exposing additional, arguably more informative, derived quantities beyond what was asked. We include this case to underscore that our lenient metric tolerates a generated query returning more information than the reference asks for, within the row it is matched against.

H. Identifier Case-Sensitivity Defect

A fourth defect, distinct from the three analyzed above, affected table-name resolution rather than result scoring. The reference SQL shipped with the Geography and ATIS benchmarks capitalizes table names as a stylistic convention of the original text2sql-data corpus (e.g., FROM STATE AS STATEalias0), independent of how those tables are actually named in any given database instance. On Linux-hosted MySQL and MariaDB, table identifiers are case-sensitive by default, and our Docker-provisioned databases created these tables with lower-case names (state, city, and so on). The immediate symptom was severe: every reference query for the Geography and ATIS schemas failed outright with a "table does not exist" error, which, combined with the vacuous-match defect described in Section V-C initially produced a reported accuracy near zero for both models on both schemas, masking any valuable information entirely. The defect was caught only by manually re-executing a handful of reference queries directly against the database and observing that even the supposedly correct SQL failed to run. The fix normalizes table-name casing against the live schema (queried via information schema) before executing either the reference or the generated query, applied identically and only to whole-word table identifiers so that column names and query aliases are left untouched. We highlight this defect specifically because, unlike the three metric-level issues discussed above, it demonstrates that even the reference ("gold") side of an execution-accuracy pipeline must be validated against the live target environment.

VI. LIMITATIONS & FUTURE WORK

- **Minimal in-context guidance:** each prompt includes exactly one worked example per schema; richer few-shot prompting or retrieval-augmented example selection would likely raise absolute accuracy for both models and could plausibly narrow or widen the gap between them.
- **Quantization:** Qwen was evaluated only at 4-bit precision, the level required to fit within a free-tier T4 GPU; full-precision inference might reduce its execution-error rate, though we would not expect it to close the observed accuracy gap given that many failures are join-path or predicate errors rather than syntax errors.
- **Custom-query subset size:** although we deliberately include all 31 custom queries rather than a proportionally down-sampled subset, 5-15 items per schema is still small enough that the per-schema custom-query percentages in Table IV carry wide confidence intervals and should be read as indicative rather than conclusive.
- **Heuristic difficulty labeling:** our automatic easy/medium/hard labels are derived from structural SQL complexity (join count, subquery nesting, aggregation) rather than validated human judgment or measured model difficulty; although the tiers order monotonically in Fig. 4, the labels remain unvalidated against measured model difficulty, and a learned or human-annotated difficulty measure would be a natural improvement.
- **Column-identity-blind lenient matching:** our lenient metric checks whether the values of a reference row appear together within some generated row, without regard to which column each value came from. This is intentional as LLMs rarely reproduce a reference query's exact column aliases even when semantically correct (e.g., population AS pop versus population), but it does admit a theoretical false-positive: two unrelated columns that

coincidentally share a numeric value (e.g., an id and an unrelated length) could be matched. We consider a semantically column-aware version of this metric a well-motivated but non-trivial direction for future work, since it would require resolving column correspondence rather than simply comparing value sets.

- **Lenient metric does not penalize extraneous rows:** as defined in Section II-B, our lenient matching is one-directional meaning it requires every reference row to be matched within the generated result, but places no requirement on generated rows that go unmatched. Concretely, if the reference result is {Athens} and a generated query returns {Athens, London, Paris}, the match still succeeds under our current implementation, even though the generated query has additionally returned incorrect, unrequested rows. We consider this a genuine limitation of the metric as implemented rather than a merely theoretical concern. A possible symmetric variant additionally requiring an injective matching in the reverse direction, from generated rows into reference rows, would close this gap and is the most direct extension of the present lenient metric for future work.
- **Single-question duplication:** post-hoc auditing of the 331-item sample identified one pair of rows sharing an identical question text and reference SQL ("what are the flights from BOSTON to BALTIMORE", present twice in the underlying ATIS corpus across different original data splits); this reduces the sample to 330 effectively distinct questions, a negligible effect on the reported statistics.
- **Prompt sensitivity:** we did not systematically vary prompt phrasing, schema-serialization format, or system-message wording; execution accuracy for both models is known in the broader literature to be sensitive to such surface-level prompt design choices, and our reported figures should be read as characterizing one reasonable prompting protocol applied identically to both models rather than either model's ceiling performance.
- **Single-run measurement:** because both models are queried at temperature zero, each question is evaluated with a single generation rather than averaged over multiple samples; this maximizes reproducibility but means our accuracy figures do not capture any run-to-run variance a non-deterministic decoding setting might introduce.
- **Schema diversity:** our three source schemas, while structurally varied (7 to 25 tables, spanning geographic, transactional-travel, and academic-records domains), are all English-language, moderately sized relational schemas drawn from an academic benchmarking tradition; we make no claim that our findings transfer to substantially larger enterprise schemas, non-English natural-language input, or NoSQL/analytical query targets outside standard SQL.

VII. USE OF AI TOOLS IN THIS WORK

In the interest of full transparency, and in accordance with the course's academic integrity requirements, we disclose the following uses of an AI-based coding and writing assistant (Anthropic's Claude) throughout this project:

- **Environment setup guidance:** step-by-step assistance configuring the Docker-based MySQL and MariaDB

containers, DBeaver connections, and the Google Colab GPU environment used to run Qwen2.5-Coder-7B.

- **Pipeline implementation:** iterative drafting of the Python modules that constitute the evaluation harness (data loading, custom-query authoring, LLM client wrappers, database execution and comparison logic, sampling, and statistical analysis), written interactively with the assistant and reviewed, run, and verified against real data and real API/database calls by the authors at every step.
- **Debugging and defect diagnosis:** the assistant helped isolate and explain the root causes of several execution-accuracy defects described in Section V including the table-name case-sensitivity mismatch, the MariaDB-incompatible session variable, and the lenient-metric vacuous-match defect by reasoning through suspicious result patterns together with the authors and proposing targeted, minimal fixes that the authors then tested directly against the databases.
- **Manuscript drafting:** prose in this paper was drafted with AI assistance and then reviewed, corrected, and approved by the authors; all reported numbers were computed by code the authors ran themselves against live databases and live model APIs, not generated or estimated by the assistant.

No experimental result, statistical figure, or conclusion in this paper was generated, fabricated, or altered by an AI tool; the assistant's role was limited to implementation support, debugging assistance, and editorial drafting, with all substantive experimental design, data verification, and interpretation decisions made and approved by the authors.

VIII. CODE AND DATA AVAILABILITY

The complete source code, raw and processed datasets, evaluation results, statistical analysis scripts, and generated figures for this project are publicly available at:

https://github.com/ntua-el21669/information_systems_project

The repository README documents the full environment setup (Docker containers, Python dependencies, API keys), the exact sequence of scripts required to reproduce every number reported in this paper, and the location of every intermediate artifact. As required, the code corresponding to the results in this submission is frozen as of the submission date and will not be modified prior to grading. The reproduction sequence is, in outline:

- **data_loader.py, custom_queries.py, summary.py** — build the 10,575-item combined corpus from the three public benchmarks plus the 31 custom queries.
- **run_experiment.py** — draw the 331-item sample and run GPT-4o-mini against MySQL (one live API pass).
- **rescore_against_db.py** — re-execute the same GPT-generated SQL against MariaDB (no new API calls).
- **export_sample_for_qwen.py, notebooks/qwen_text2sql_setup.ipynb** — export the identical 331-item sample with schema context for the Colab notebook, and run Qwen2.5-Coder-7B inference there (one live GPU inference pass).

- **score_qwen_results.py, rescore_against_db.py** — score the Qwen output against MySQL and re-score against MariaDB.

- **analyze_results.py** — compute Wilson confidence intervals, the paired McNemar tests, and generate Fig. 2–4 and Tables I–IV from the four resulting result files.

This sequence requires only two live model-inference passes (one GPT API call sequence, one Qwen GPU inference run) to populate all four model-database combinations, since the generated SQL for a given model is re-executed rather than regenerated when switching between MySQL and MariaDB.

IX. CONCLUSION

This paper presented LLMSQL3, an empirical evaluation of Text-to-SQL semantic parsing comparing GPT-4o-mini and Qwen2.5-Coder-7B-Instruct across MySQL and MariaDB. Using a stratified 331-item sample that deliberately preserves full coverage of a custom-designed set of 31 complex queries, and a rigorous paired statistical protocol (exact McNemar tests, Wilson confidence intervals), we find that GPT-4o-mini significantly outperforms Qwen2.5-Coder-7B-Instruct under both a strict ($p = 0.0161$) and a lenient ($p = 0.0060$) execution-accuracy metric, while the choice of RDBMS engine has a negligible effect on accuracy but a measurable effect on which individual reference queries execute successfully. Beyond the headline comparison, we document several concrete implementation pitfalls in execution-accuracy evaluation — identifier case-sensitivity, RDBMS-specific session variables, and vacuous empty-result matching — together with the methodological journey by which the least defensible of our early design choices was identified and corrected. We believe this process-level account is of comparable value to the headline numbers themselves, since it is precisely the kind of defect that a self-consistent but under-scrutinized pipeline will not surface on its own.

Future work will investigate richer few-shot and retrieval-augmented prompting for both models, full-precision (non-quantized) inference for the open-weights model to isolate the effect of quantization from the effect of raw model scale, and a column-identity-aware variant of the lenient metric that resolves value-to-column correspondence rather than treating each row as an unordered set of values. A further natural extension is to repeat this protocol on a larger, non-legacy benchmark such as Spider or BIRD, where reference queries are less likely to carry the historical, corpus-specific idiosyncrasies we documented for ATIS in Section V-B.

ACKNOWLEDGMENT

The authors acknowledge the maintainers of the text2sql-data repository and the developers of the Qwen and OpenAI foundation models used in this study.

REFERENCES

- [1] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, "Natural language interfaces to databases—an introduction," *Nat. Lang. Eng.*, vol. 1, no. 1, pp. 29–81, 1995.
- [2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Raděv, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [3] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv:1709.00103*, 2017.

- [4] N. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. NeurIPS*, vol. 36, 2023, pp. 34015–34027.
- [5] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, S. Li, K. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs (BIRD)," in *Proc. NeurIPS*, vol. 36, 2023.
- [6] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proc. AAAI*, 1996, pp. 1050–1055.
- [7] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington, "The ATIS spoken language systems pilot corpus," in *Proc. DARPA Speech and Natural Language Workshop*, 1990, pp. 96–101.
- [8] J. K. Kummerfeld, S. Joshi, and X. V. Lin, "text2sql-data: Standardized datasets for text-to-SQL research," *GitHub Repository*, 2020.
- [9] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder Technical Report," *arXiv:2409.12186*, 2024.
- [10] OpenAI, "GPT-4o System Card," *arXiv:2410.21276*, 2024.
- [11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [12] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.